

## Unit-No.4

### Functional Dependencies:-

- A functional dependency (FD) is a relationship between two attributes, typically between the PK and other non-key attributes within a table.
- For any relation R, attribute Y is functionally dependent on attribute X (usually the PK), if for every valid instance of X, that value of X uniquely determines the value of Y.
- This relationship is indicated by the representation below :

**X -----> Y**

- The left side of the above FD diagram is called the determinant, and the right side is the dependent. Here are a few examples.
- In the first example, below, SIN determines Name, Address and Birthdate. Given SIN, we can determine any of the other attributes within the table.
- **SIN -----> Name, Address, Birthdate**
- For the second example, SIN and Course determine the date completed (DateCompleted). This must also work for a composite PK.
- **SIN, Course ----> DateCompleted**

### Types of Functional dependencies in DBMS:

Trivial functional dependency

Non-Trivial functional dependency

Multivalued functional dependency

Transitive functional dependency

## 1. Trivial Functional Dependency

In **Trivial Functional Dependency**, a dependent is always a subset of the determinant.

i.e. If  $X \rightarrow Y$  and **Y is the subset of X**, then it is called trivial functional dependency

| roll_no | name | age |
|---------|------|-----|
| 42      | abc  | 17  |
| 43      | pqr  | 18  |
| 44      | xyz  | 18  |

Here,  $\{\text{roll\_no, name}\} \rightarrow \text{name}$  is a trivial functional dependency, since the dependent **name** is a subset of determinant set **{roll\_no, name}**

Similarly,  $\text{roll\_no} \rightarrow \text{roll\_no}$  is also an example of trivial functional dependency.

## 2. Non-trivial Functional Dependency

- In **Non-trivial functional dependency**, the dependent is strictly not a subset of the determinant.

i.e. If  $X \rightarrow Y$  and **Y is not a subset of X**, then it is called Non-trivial functional dependency.

| roll_no | name | age |
|---------|------|-----|
| 42      | abc  | 17  |
| 43      | pqr  | 18  |
| 44      | xyz  | 18  |

Here, **roll\_no**  $\rightarrow$  **name** is a non-trivial functional dependency, since the dependent **name** is **not a subset of** determinant **roll\_no**

Similarly, **{roll\_no, name}**  $\rightarrow$  **age** is also a non-trivial functional dependency, since **age** is **not a subset of** **{roll\_no, name}**

## 2. Multivalued Functional Dependency

- In **Multivalued functional dependency**, entities of the dependent set are **not dependent on each other**.  
i.e. If **a**  $\rightarrow$  **{b, c}** and there exists **no functional dependency** between **b** and **c**, then it is called a **multivalued functional dependency**.

| roll_no | name | age |
|---------|------|-----|
| 42      | abc  | 17  |
| 43      | pqr  | 18  |

|    |     |    |
|----|-----|----|
| 44 | xyz | 18 |
| 45 | abc | 19 |

Here,  $\text{roll\_no} \rightarrow \{\text{name}, \text{age}\}$  is a multivalued functional dependency, since the dependents **name** & **age** are **not dependent** on each other (i.e. **name**  $\rightarrow$  **age** or **age**  $\rightarrow$  **name** doesn't exist !)

### 3. Transitive Functional Dependency

In transitive functional dependency, dependent is indirectly dependent on determinant.

i.e. If  $a \rightarrow b$  &  $b \rightarrow c$ , then according to axiom of transitivity,  $a \rightarrow c$ . This is a **transitive functional dependency**

|    |     |    |   |
|----|-----|----|---|
| 42 | abc | CO | 4 |
| 43 | pqr | EC | 2 |
| 44 | xyz | IT | 1 |
| 45 | abc | EC | 2 |
| 42 | abc | CO | 4 |

Here, **enrol\_no**  $\rightarrow$  **dept** and **dept**  $\rightarrow$  **building\_no**,

Hence, according to the axiom of transitivity, **enrol\_no**  $\rightarrow$  **building\_no** is a valid functional dependency.

This is an indirect functional dependency, hence called Transitive functional dependency.

### Closures of a set of functional dependencies

- A **Closure** is a set of FDs is a set of all possible FDs that can be derived from a given set of FDs. It is also referred as a **Complete** set of FDs. If F is used to denote the set of FDs for relation R, then a closure of a set of FDs implied by F is denoted by  $F^+$ .
- Let's consider the set F of functional dependencies given below:  
 **$F = \{A \rightarrow B, B \rightarrow C, C \rightarrow D\}$**   
from F, it is possible to derive following dependencies.
- - A  $\rightarrow$  A ...By using Rule-4, Self-Determination.
  - A  $\rightarrow$  B ...Already given in F.
  - A  $\rightarrow$  C ...By using rule-3, Transitivity.
  - A  $\rightarrow$  D ...By using rule-3, Transitivity.
- Now, by applying Rule-6 Union, it is possible to derive  $A^+ \rightarrow ABCD$  and it can be denoted using  $A \rightarrow ABCD$ .
- All such type of FDs derived from each FD of F form a closure of F. Steps to determine  $F^+$  example:
- Determine each set of attributes **X** that appears as a left hand side of some FD in F.
- Determine the set  $X^+$  of all attributes that are dependent on **X**, as given in above example.
- In other words,  $X^+$  represents a set of attributes that are functionally determined by X based on F. And,  $X^+$  is called the **Closure of X under F**.
- All such sets of  $X^+$ , in combine, Form a closure of F.

### Normalization:-

- A large database defined as a single relation may result in data duplication. This repetition of data may result in:
  - Making relations very large.
  - It isn't easy to maintain and update data as it would involve searching many records in relation.
  - Wastage and poor utilization of disk space and resources.
  - The likelihood of errors and inconsistencies increases.

### **What is Normal Form in DBMS?**

- The process of organising information/data in a database is known as normalization. This process helps us minimise the redundancy of data from a set of relations or a single relation. It also helps us eliminate several undesirable characteristics, such as Update, Insertion, and Deleting of Anomalies.

### **Anomalies in a Database**

- When the design of a database isn't perfect, it may consist of anomalies. These anomalies can be a nightmare for database administrators. It is next to impossible to manage any database that has these anomalies. Let us discuss a few of these:
  - **Update Anomalies:** When the data elements/ items are scattered and aren't really linked properly to each other, it may lead to some strange situations. For instance, if we try updating a data element with its copies scattered all over various places, then only a few instances will be updated properly. On the other hand, the rest others will be left with their old values. These types of instances will always leave a database in a state that's pretty inconsistent.
  - **Deletion Anomalies:** When we try to delete any record, a few parts of it are left undeleted due to unawareness, then the info/ data will get stored at some separate place.
  - **Insert Anomalies:** It happens when we try inserting data in some non-existent record.

### **Types of Normal Forms**

- **Normal forms** are used to eliminate or reduce redundancy in database tables.
- 1NF
- 2NF
- 3NF
- BCNF
- 4NF
- 5NF

### **1. First Normal Form –**

- If a relation contains composite or multi-valued attribute, it violates first normal form or a relation is in first normal form if it does not contain any composite or multi-valued attribute.
- A relation is in first normal form if every attribute in that relation is **singled valued attribute**.
- **Example 1** – Relation STUDENT in table 1 is not in 1NF because of multi-valued attribute STUD\_PHONE. Its decomposition into 1NF has been shown in table 2.

| STUD_NO | STUD_NAME | STUD_PHONE                | STUD_STATE | STUD_COUNTRY |
|---------|-----------|---------------------------|------------|--------------|
| 1       | RAM       | 9716271721,<br>9871717178 | HARYANA    | INDIA        |
| 2       | RAM       | 9898297281                | PUNJAB     | INDIA        |
| 3       | SURESH    |                           | PUNJAB     | INDIA        |

Table 1

Conversion to first normal form

| STUD_NO | STUD_NAME | STUD_PHONE | STUD_STATE | STUD_COUNTRY |
|---------|-----------|------------|------------|--------------|
| 1       | RAM       | 9716271721 | HARYANA    | INDIA        |
| 1       | RAM       | 9871717178 | HARYANA    | INDIA        |
| 2       | RAM       | 9898297281 | PUNJAB     | INDIA        |
| 3       | SURESH    |            | PUNJAB     | INDIA        |

Table 2

## 2) Second normal form (2NF)

- A table is said to be in 2NF if both the following conditions hold:
- Table is in 1NF (First normal form)
- No non-prime attribute is dependent on the proper subset of any **candidate key** of table.
- **An attribute that is not part of any candidate key is known as non-prime attribute.**
- **Example:** Let's say a school wants to store the data of teachers and the subjects they teach. They create a table Teacher that looks like this: Since a teacher can teach more than one subjects, the table can have multiple rows for a same teacher.

| Teacher_Id | Subject   | Teacher_Age |
|------------|-----------|-------------|
| 111        | Maths     | 38          |
| 111        | Physics   | 38          |
| 222        | Biology   | 38          |
| 333        | Physics   | 40          |
| 333        | Chemistry | 40          |

- **Candidate Keys:** {Teacher\_Id, Subject}  
**Non prime attribute:** Teacher\_Age
- This table is in 1 NF because each attribute has atomic values. However, it is not in 2NF because non prime attribute Teacher Age is dependent on Teacher\_Id alone which is a proper subset of candidate key.
- This violates the rule for 2NF as the rule says "**no non-prime attribute is dependent on the proper subset of any candidate key of the table**".

To make the table complies with 2NF we can disintegrate it in two tables like this:

Teacher\_Details table:

| Teacher_Id | Teacher_Age |
|------------|-------------|
| 111        | 38          |
| 222        | 38          |
| 333        | 40          |

Teacher\_Subject table:

| Teacher_Id | Subject   |
|------------|-----------|
| 111        | Maths     |
| 111        | Physics   |
| 222        | Biology   |
| 333        | Physics   |
| 333        | Chemistry |

Now the tables are in Second normal form (2NF). To learn more about 2NF refer this guide: [2NF](#)

### 3) Third Normal form (3NF):-

- A table design is said to be in 3NF if both the following conditions hold:
- Table must be in 2NF
- **Transitive functional dependency** of non-prime attribute on any super key should be removed.
- An attribute that is not part of any **candidate key** is known as non-prime attribute.
- In other words 3NF can be explained like this: A table is in 3NF if it is in 2NF and for each functional dependency  $X \rightarrow Y$  at least one of the following conditions hold:
  - X is a **super key** of table
  - Y is a prime attribute of table
  - An attribute that is a part of one of the candidate keys is known as prime attribute.

Example: Let's say a company wants to store the complete address of each employee, they create a table named Employee\_Details that looks like this:

| Emp_Id | Emp_Name | Emp_Zip | Emp_State | Emp_City | Emp_District   |
|--------|----------|---------|-----------|----------|----------------|
| 1001   | John     | 282005  | UP        | Agra     | Dayal Bagh     |
| 1002   | Ajeet    | 222008  | TN        | Chennai  | M-City         |
| 1006   | Lora     | 282007  | TN        | Chennai  | Urrapakka<br>m |
| 1101   | Lilly    | 292008  | UK        | Pauri    | Bhagwan        |
| 1201   | Steve    | 222999  | MP        | Gwalior  | Ratan          |

- **Super keys:** {Emp\_Id}, {Emp\_Id, Emp\_Name}, {Emp\_Id, Emp\_Name, Emp\_Zip}...so on
- 
- **Candidate Keys:** {Emp\_Id}
- 
- **Non-prime attributes:** all attributes except Emp\_Id are non-prime as they are not part of any candidate keys.
- Here, Emp\_State, Emp\_City & Emp\_District dependent on Emp\_Zip. Further Emp\_zip is dependent on Emp\_Id that makes non-prime attributes (Emp\_State, Emp\_City & Emp\_District) transitively dependent on super key (Emp\_Id). This violates the rule of 3NF.
- To make this table complies with 3NF we have to disintegrate the table into two tables to remove the transitive dependency:

Employee Table:

| Emp_Id | Emp_Name | Emp_Zip |
|--------|----------|---------|
| 1001   | John     | 282005  |
| 1002   | Ajeet    | 222008  |
| 1006   | Lora     | 282007  |
| 1101   | Lilly    | 292008  |
| 1201   | Steve    | 222999  |

Employee\_Zip table:

| Emp_Zip | Emp_State | Emp_City | Emp_District |
|---------|-----------|----------|--------------|
| 282005  | UP        | Agra     | Dayal Bagh   |
| 222008  | TN        | Chennai  | M-City       |
| 282007  | TN        | Chennai  | Urrapakkam   |
| 292008  | UK        | Pauri    | Bhagwan      |
| 222999  | MP        | Gwalior  | Ratan        |

#### 4) Boyce Codd normal form (BCNF):-

- It is an advance version of 3NF that's why it is also referred as 3.5NF. BCNF is stricter than 3NF.
- A table complies with BCNF if it is in 3NF and for every **functional dependency**  $X \rightarrow Y$ , X should be the super key of the table.
- **Example:** Suppose there is a company wherein employees work in **more than one department**.

They store the data like this:

| Emp_Id | Emp_Nationality | Emp_Dept                     | Dept_Type | Dept_No_Of_Emp |
|--------|-----------------|------------------------------|-----------|----------------|
| 1001   | Austrian        | Production and planning      | D001      | 200            |
| 1001   | Austrian        | stores                       | D001      | 250            |
| 1002   | American        | design and technical support | D134      | 100            |
| 1002   | American        | Purchasing department        | D134      | 600            |

- **Functional dependencies in the table above:**  
Emp\_Id -> Emp\_Nationality  
Emp\_Dept -> {Dept\_Type, Dept\_No\_Of\_Emp}
- **Candidate key:** {Emp\_Id, Emp\_Dept}
- The table is not in BCNF as neither Emp\_Id nor Emp\_Dept alone are keys.
- To make the table comply with BCNF we can break the table in three tables like this:

Emp\_Nationality table:

| Emp_Id | Emp_Nationality |
|--------|-----------------|
| 1001   | Austrian        |
| 1002   | American        |

Emp\_Dept table:

| Emp_Dept                     | Dept_Type | Dept_No_Of_Emp |
|------------------------------|-----------|----------------|
| Production and planning      | D001      | 200            |
| stores                       | D001      | 250            |
| design and technical support | D134      | 100            |
| Purchasing department        | D134      | 600            |

Emp\_Dept\_Mapping table:

| Emp_Id | Emp_Dept                     |
|--------|------------------------------|
| 1001   | Production and planning      |
| 1001   | stores                       |
| 1002   | design and technical support |
| 1002   | Purchasing department        |

- **Functional dependencies:**

Emp\_Id -> Emp\_Nationality

Emp\_Dept -> {Dept\_Type, Dept\_No\_Of\_Emp}

- **Candidate keys:**

For first table: Emp\_Id

For second table: Emp\_Dept

For third table: {Emp\_Id, Emp\_Dept}

- This table is now in BCNF as in both the functional dependencies left side part is a key.

### 5) Fourth normal form (4NF)

- A relation will be in 4NF if it is in Boyce Codd normal form and has no multi-valued dependency.
- For a dependency  $A \twoheadrightarrow B$ , if for a single value of A, multiple values of B exists, then the relation will be a multi-valued dependency.

Example

| STU_ID | COURSE    | HOBBY   |
|--------|-----------|---------|
| 21     | Computer  | Dancing |
| 21     | Math      | Singing |
| 34     | Chemistry | Dancing |
| 74     | Biology   | Cricket |
| 59     | Physics   | Hockey  |

- The given STUDENT table is in 3NF, but the COURSE and HOBBY are two independent entity. Hence, there is no relationship between COURSE and HOBBY.
- In the STUDENT relation, a student with STU\_ID, **21** contains two courses, **Computer** and **Math** and two hobbies, **Dancing** and **Singing**. So there is a Multi-valued dependency on STU\_ID, which leads to unnecessary repetition of data.

So to make the above table into 4NF, we can decompose it into two tables:  
**STUDENT\_COURSE**

| STU_ID | COURSE    |
|--------|-----------|
| 21     | Computer  |
| 21     | Math      |
| 34     | Chemistry |
| 74     | Biology   |
| 59     | Physics   |

## **STUDENT\_HOBBY**

| STU_ID | HOBBY   |
|--------|---------|
| 21     | Dancing |
| 21     | Singing |
| 34     | Dancing |
| 74     | Cricket |
| 59     | Hockey  |

### **6)Fifth normal form (5NF):-**

- A relation is in 5NF if it is in 4NF and not contains any join dependency and joining should be lossless.
- 5NF is satisfied when all the tables are broken into as many tables as possible in order to avoid redundancy.

- 5NF is also known as Project-join normal form (PJ/NF).

**Example:-**

| SUBJECT   | LECTURER | SEMESTER   |
|-----------|----------|------------|
| Computer  | Anshika  | Semester 1 |
| Computer  | John     | Semester 1 |
| Math      | John     | Semester 1 |
| Math      | Akash    | Semester 2 |
| Chemistry | Praveen  | Semester 1 |

- In the above table, John takes both Computer and Math class for Semester 1 but he doesn't take Math class for Semester 2. In this case, combination of all these fields required to identify a valid data.
- Suppose we add a new Semester as Semester 3 but do not know about the subject and who will be taking that subject so we leave Lecturer and Subject as NULL. But all three columns together acts as a primary key, so we can't leave other two columns blank.

So to make the above table into 5NF, we can decompose it into three relations P1, P2 & P3:

P1

| SEMESTER   | SUBJECT   |
|------------|-----------|
| Semester 1 | Computer  |
| Semester 1 | Math      |
| Semester 1 | Chemistry |
| Semester 2 | Math      |

P2

| SUBJECT   | LECTURER |
|-----------|----------|
| Computer  | Anshika  |
| Computer  | John     |
| Math      | John     |
| Math      | Akash    |
| Chemistry | Praveen  |